

Le RL — ce qui fonctionne, ce qui résiste, et l'art de s'en servir

Maîtriser l'apprentissage par renforcement (RL), ce n'est pas connaître *plus* d'algorithmes. C'est savoir **quand il marche, pourquoi il échoue, et comment le rattraper**. Ce dossier est cette carte — en clair, sans bagage mathématique.

Rappel en une phrase : le RL apprend **par essai et erreur**, guidé par une simple **récompense**. On ne lui montre jamais la bonne réponse ; il la découvre. C'est sa force — et la source de tous ses ennuis.

1. Là où le RL excelle (ses terrains de prédilection)

Le RL est le bon outil quand **quatre conditions** sont réunies :

1. Il existe un **but mesurable** (une récompense), mais **pas de « bonnes réponses »** à montrer.
2. On peut **interagir énormément et à bas coût** (un simulateur, un jeu).
3. Le problème est **séquentiel** : chaque geste influence la suite.
4. On veut **optimiser fort** un objectif bien défini, parfois au-delà de l'humain.

Quand ces conditions sont là, les résultats sont spectaculaires :

Réussite	Domaine
DQN sur Atari, AlphaGo / AlphaZero	jeux
AlphaStar (StarCraft II), OpenAI Five (Dota)	jeux complexes, temps réel
Locomotion simulée (MuJoCo), ballons stratosphériques (Loon)	contrôle
Placement de circuits (AlphaChip), refroidissement de datacenters	optimisation industrielle
Ajustement des grands modèles de langage (RLHF)	l'IA qui te répond

Le fil commun : un objectif clair + une interaction massive et bon marché. **Ton projet coche les cases** (mission = récompense, Arma = simulateur).

2. Là où le RL résiste (les murs, en clair)

C'est la partie la plus importante : les **modes d'échec** du RL. Les connaître, c'est déjà à moitié les éviter.

① **La faim de données.** Le RL réclame des millions, parfois des milliards d'essais. Il échoue dès que l'interaction est rare, lente, chère ou dangereuse (un vrai robot, un vrai champ de bataille). *Image* : un apprenti qui n'apprend qu'en se trompant — combien de chutes faut-il, et peux-tu te les permettre ?

② **Le détournement de récompense.** L'agent optimise la *lettre* de la récompense, pas son *esprit*. Exemple célèbre : un bateau de course qui, au lieu de finir la course, tourne en rond pour ramasser indéfiniment des bonus. *Image* : tu paies au nombre de lignes de code écrites — tu obtiens du code verbeux, pas du bon code.

③ **La récompense rare.** Si le succès n'arrive presque jamais (« mission réussie » une fois sur mille), l'agent erre à l'aveugle, sans savoir quel geste, mille pas plus tôt, a compté. *Image* : apprendre à jouer aux échecs en n'apprenant que le résultat final, jamais la valeur d'un coup.

④ **L'exploration.** Comment découvrir un bon comportement qu'on n'a jamais vu ? Sans incitation à explorer, l'agent reste dans ce qu'il connaît. *Image* : on ne trouve pas une porte cachée si l'on ne cherche jamais derrière les murs.

⑤ **La fragilité.** Le RL est réputé capricieux : un simple changement de graine aléatoire ou d'un réglage peut donner deux résultats opposés. Deux entraînements « identiques », deux issues. *Image* : une recette qui réussit un jour et rate le lendemain, sans qu'on sache pourquoi.

⑥ **La cible mouvante (multi-agents).** Quand les autres agents apprennent aussi, le sol bouge sous les pieds de chacun : ce qui marchait hier ne marche plus. La propriété de stabilité que suppose le RL classique est rompue.

⑦ **Le sur-apprentissage de l'environnement.** L'agent maîtrise *la* carte d'entraînement et s'effondre sur une carte nouvelle. Il a mémorisé, pas compris. C'est le fameux « fossé simulation → réalité ».

⑧ **Pas de bon sens.** L'agent ne connaît *que* sa récompense. En dehors d'elle, aucune intuition, aucune morale, aucune prudence — sauf si tu les y as mises explicitement.

3. L'art : comment les experts font marcher le RL

À chaque mur, un (ou plusieurs) remède. **C'est ça, maîtriser le RL** : reconnaître le mur, et appliquer le bon remède.

Le mur	Le remède
① Faim de données	Simulateur rapide ; mettre au point sur un simulateur-jouet d'abord ; réutiliser l'expérience (<i>replay</i>) ; apprentissage par modèle ; parallélisme
② Détournement	Façonnage par potentiel uniquement ; surveiller le comportement réel ; apprendre la récompense par imitation/IRL ; poser des contraintes
③ Récompense rare	Curriculum (du facile au dur) ; motivation intrinsèque (curiosité) ; réétiquetage des échecs (HER) ; machines à récompense
④ Exploration	Récompense de curiosité (RND) ; bruit structuré ; auto-jeu
⑤ Fragilité	Partir d'une base solide (PPO) ; régler soigneusement ; plusieurs graines ; suivi rigoureux (TensorBoard/W&B)
⑥ Cible mouvante	CTDE ; auto-jeu discipliné (jeu fictif, ligue)
⑦ Sur-apprentissage	Randomisation des scénarios ; multi-missions ; curriculum automatique (PLR)
⑧ Pas de bon sens / sécurité	CMDP / safe RL ; contraintes dures ; supervision humaine

Le réflexe maître : ne jamais faire confiance aveuglément au comportement appris. Toujours **regarder** ce que l'agent fait réellement (en visuel, sur des cas), car c'est là qu'on attrape les détournements de récompense avant qu'ils ne pourrissent tout.

4. La question d'or : faut-il vraiment du RL ici ?

Un expert se la pose **avant** de coder. Le RL est souvent un **dernier recours**, pas un premier réflexe. Compare honnêtement :

Si...	...préfère
Tu as des exemples de bon comportement (démonstrations)	l' imitation (clonage, IRL) — bien plus simple et stable
Le problème a une solution connue / calculable	la planification ou l' optimisation classique
Le comportement est simple et descriptible	un script / arbre de comportement (BT)
Tu as une récompense mais aucune étiquette , c'est séquentiel , et tu peux interagir beaucoup	le RL

Pour du combat, la réponse experte est rarement « tout en RL ». C'est un **hybride** : un arbre de comportement doctrinal pour le « comment » (se mettre à couvert, recharger), et le RL pour le « quand » et le « qui » (Module 7). On apprend ce qui est dur à scripter, on script ce qui est dur à apprendre.

5. Ce que tout cela signifie pour ton projet

Surprise utile : **notre plan complet n'est rien d'autre qu'un catalogue des remèdes ci-dessus, appliqués à ton cas.**

- Ta contrainte qui domine tout = **faim de données × lenteur d'Arma** → simulateur-jouet d'abord, abstraction des actions, hybride BT, amorçage par imitation. (Murs ①)
- Ta récompense de mission est **rare** et le sim est **riche** → machines à récompense + façonnage par potentiel **seulement**, pour éviter le détournement. (Murs ②③)
- Tu veux des agents **qui généralisent** → multi-missions, PLR. (Mur ⑦)
- Tu veux de la **stabilité** → MAPPO/HAPPO, plusieurs graines. (Murs ⑤⑥)
- Tu veux qu'ils **restent en vie** → CMDP. (Mur ⑧)

Autrement dit : **on n'improvise pas. On applique l'art.** Chaque choix de notre architecture répond à un mur connu du RL.

En dernier regard. Le RL met à nu une vérité dérangeante : un agent ne devient que ce que sa récompense l'invite à devenir, et il poursuivra toujours la mesure plutôt que le sens. C'est la loi de Goodhart — « quand une mesure devient un objectif, elle cesse d'être une bonne mesure ». Tout l'art du RL tient dans cette lucidité : la récompense n'est jamais tout à fait le but ; elle n'en est qu'une ombre chiffrée. Maîtriser le RL, ce n'est donc pas dompter une machine, c'est cultiver une humilité — celle de qui sait l'écart irréductible entre ce qu'on peut mesurer et ce qu'on veut vraiment. Celui qui l'oublie fabrique des agents brillants et insensés ; celui qui s'en souvient fabrique des agents qu'on peut, enfin, laisser agir.